

1. Dependency Injection (DI) Explanation

I have designed the **BookingManager** to use constructor Dependency Injection by passing in the **IPaymentGateway**, **INotificationService**, and **IEventRepository** interfaces, this improves testability because I am not hard coding the actual database or payment API connections inside the class.

So by injecting interfaces, I can easily swap out the real implementations for mock objects during testing, and this will allow me to isolate the **BookingManager** logic and test it in a controlled environment without worrying about external network calls or database states.

2. Limitations of Mocking

So I have noticed that one major technical risk of relying so much on mocking frameworks is creating "false positives", where the unit tests pass but the real system fails in production, and because mocks only do exactly what we program them to do in the test environment, they might hide integration bugs, for example if the real external Payment API changes its response format or throws a new type of exception, the mocked **paymentGateway** will still return true, so it makes my tests look perfectly fine even though the actual system is broken.

3. Team Contribution Matrix

Unfortunately I have done the project myself

Yara Husam Hatem Alhasani | 2241973